



Unidad II

Clase 2

De la abstracción a la implementación

TDA Pila y TDA Cola

Traducir una especificación abstracta a una implementación concreta respetando el contrato.

Un **dato** es simplemente una representación de información que el programa puede almacenar y manipular.

Tipos de Datos



int

Dominio: números enteros

Operaciones: suma, resta, multiplicación, división, comparación.

String

Dominio: cadenas de texto

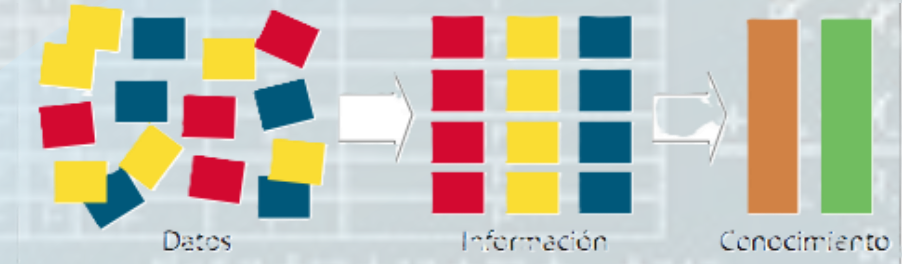
Operaciones: concatenar, comparar, obtener longitud.

boolean

Dominio: verdadero o falso

Operaciones: AND, OR, NOT.

FILOSOFÍA DE DISEÑO



Un **Tipo de Dato Abstracto (TDA)** o **Abstract Data Type (ADT)** es un modelo matemático que define un conjunto de datos y las operaciones que se pueden realizar sobre ellos, sin especificar cómo se implementan internamente



Precondición y contrato

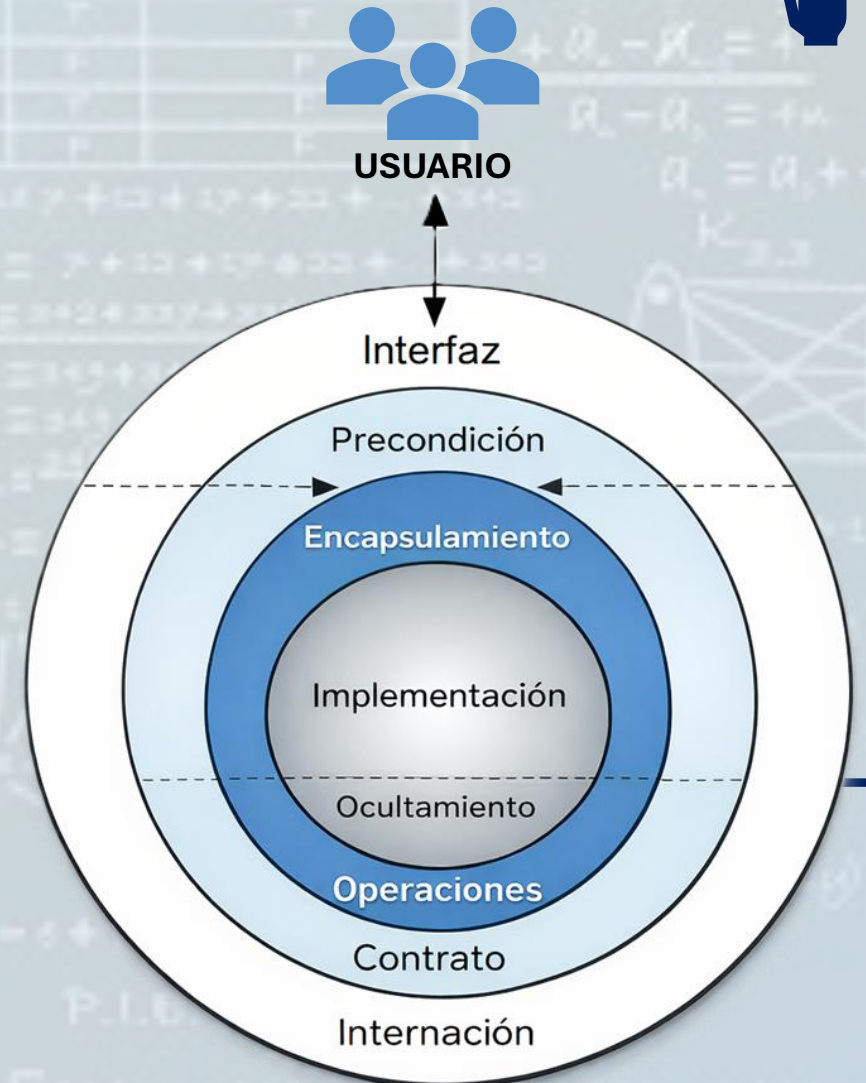


Una **precondición** es una condición que debe cumplirse antes de ejecutar una operación.

Por ejemplo, no tiene sentido quitar un elemento de una pila vacía. Por lo tanto, la operación “desapilar” requiere como precondición que la pila no esté vacía.

Pensar en **contrato** es pensar en términos profesionales. El usuario del TDA se compromete a respetar las precondiciones y por lo tanto la implementación se compromete a garantizar el comportamiento especificado si esas condiciones se cumplen.

Significa que cada módulo del sistema tiene responsabilidades claras.



¿Qué es un TDA? (Tipo de Dato Abstracto)

Tipo de Dato Abstracto (TDA)

Define estructuras de datos según:

- **Qué hace** (sin importar)
- **Cómo lo hace**

TDA

QUÉ HACE (Contrato)

- Operaciones
- Reglas
- Comportamiento

CÓMO LO HACE (Implementación)

- Estructura interna
- Algoritmos
- Variables / Memoria

El usuario usa el TDA sin conocer su implementación interna



Encapsulamiento

Protege los datos internos



Independencia

Cambia la implementación, no el uso



Reutilización

Mismo TDA, distintos contextos

Ejemplo: Pila

Usuario ve:

- `push ( )`
- `pop ( )`

Sistema hace:

- agrega en array
- mueve índice (tope)

NO necesita saber cómo funciona internamente

TDA PILA (STACK)



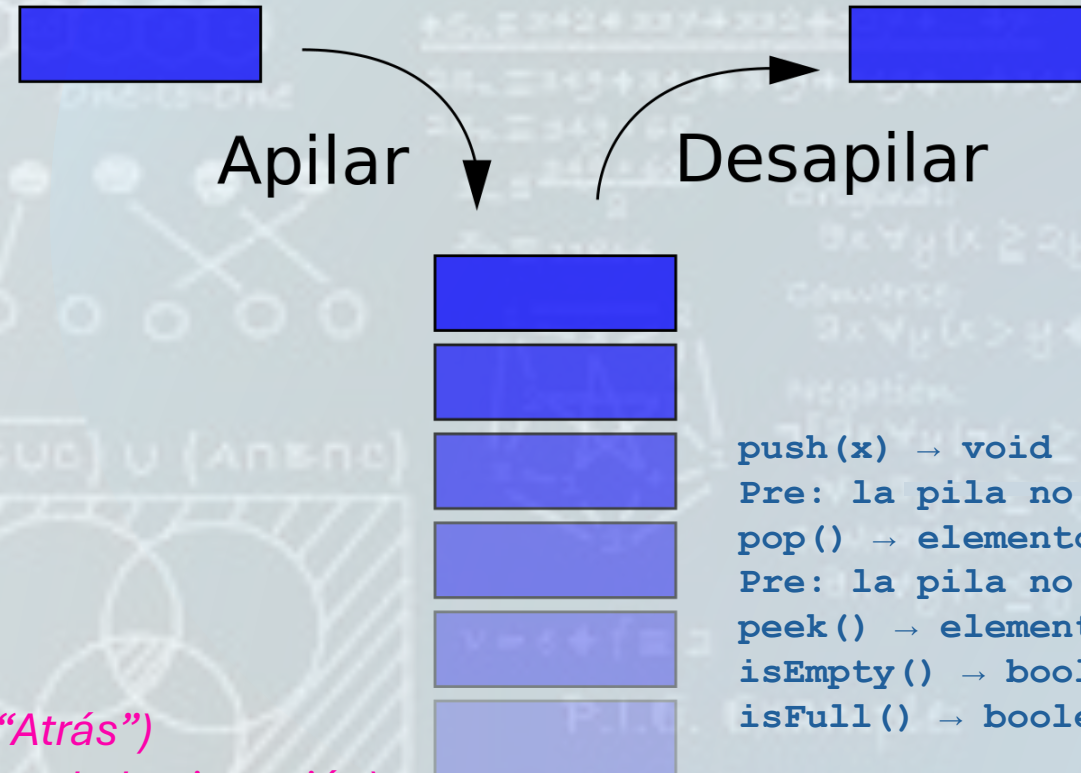
OPERACIONES BÁSICAS

- push(): apilar, meter
- pop(): desapilar, sacar
- peek(): cima, tope
- isEmpty(): es vacío
- isFull(): está lleno

PRINCIPIO

- LIFO (Last In, First Out)
Ultimo en entrar primero en salir

Una pila es una estructura de datos en la que todas las inserciones y eliminaciones se realizan por un mismo extremo denominado tope o cima de la pila



push(x) → void
 Pre: la pila no está llena
 pop() → elemento
 Pre: la pila no está vacía
 peek() → elemento
 isEmpty() → boolean
 isFull() → boolean



Una cola es una estructura de datos en la que las inserciones se realizan en un extremo llamado final, y las eliminaciones en el otro extremo llamado frente

OPERACIONES BÁSICAS

- enqueue(): agregar
- dequeue(): quitar
- front(): ver el primero
- isEmpty(): es vacío
- isFull(): está lleno



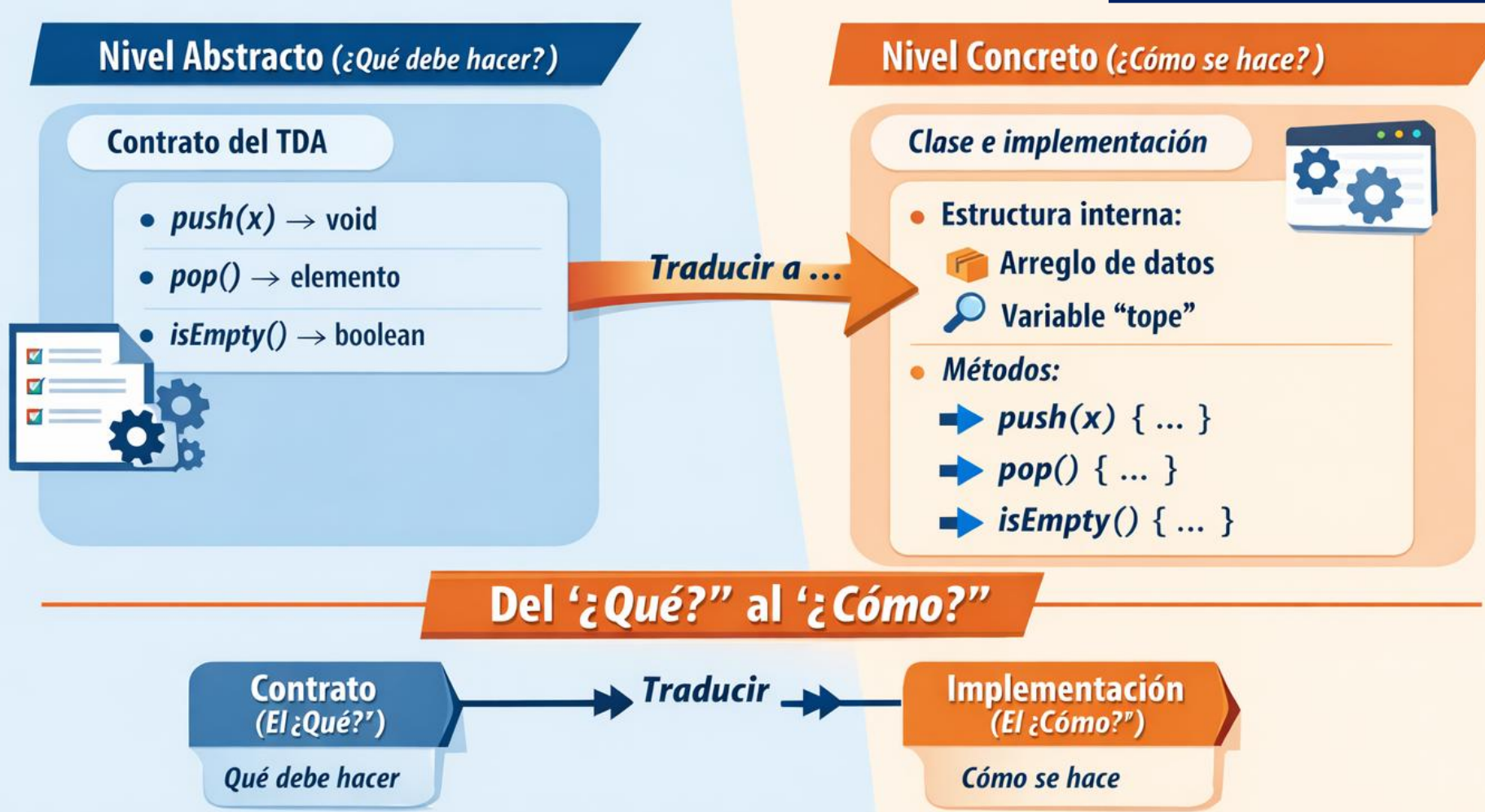
PRINCIPIO

- FIFO (First In, First Out)
Primero en entrar primero en salir

Turnos (ordenación por orden de llegada)
Procesos en sistemas (orden de ejecución de tareas)

enqueue(x) → void
Pre: la cola no está llena
dequeue() → elemento
Pre: la cola no está vacía
front() → elemento
Pre: la cola no está vacía
isEmpty() → boolean
isFull() → boolean

Del contrato a la implementación



Consiste en transformar la definición abstracta de un TDA en una solución concreta en código, donde se decide cómo representar los datos internamente y cómo programar cada operación para que cumpla exactamente con ese comportamiento.



Contrato del TDA

- `push(x) → void`
- `pop() → elemento`
- `isEmpty() → boolean`



Representación Interna



La representación interna es la forma elegida para materializar la estructura sin alterar su comportamiento abstracto.

La representación será **estática con arreglos**. Eso significa que los elementos van a almacenarse dentro de un vector de tamaño fijo.

Por ejemplo, una pila puede representarse internamente con:

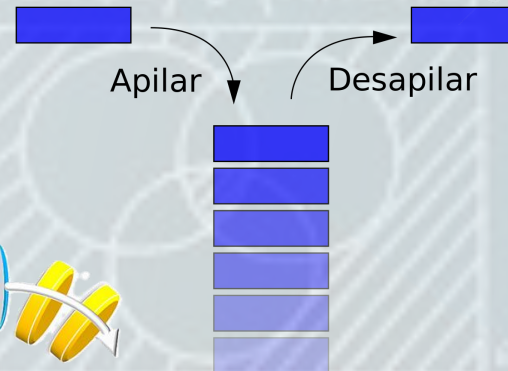
- un arreglo para guardar los elementos
- una variable que indique cuál es el tope actual

Una cola puede representarse con:

- un arreglo para guardar los elementos
- una variable para el frente
- una variable para el final



```
public class Pila {  
  
    public static void main(String[] args) {  
  
        Stack<String> pila = new Stack<String>();  
  
        for (int x=1;x≤10;x++)  
            pila.push(Integer.toString(x));  
  
        while (!pila.empty())  
            System.out.println(pila.pop());  
  
    }  
}
```



El usuario de la estructura ve operaciones como `push()` o `enqueue()`, pero la clase internamente trabaja con índices, posiciones y validaciones.



- Estructura interna:
 - Arreglo de datos
 - Variable "tope"

Estructura Interna y Variables de control



La estructura interna está compuesta por los elementos que la clase necesita para funcionar por dentro. Estos atributos no forman parte del contrato externo del TDA. Son recursos internos de la implementación

En una implementación simple de una pila con arreglo, la estructura interna podría ser:

- un arreglo datos
- un entero tope

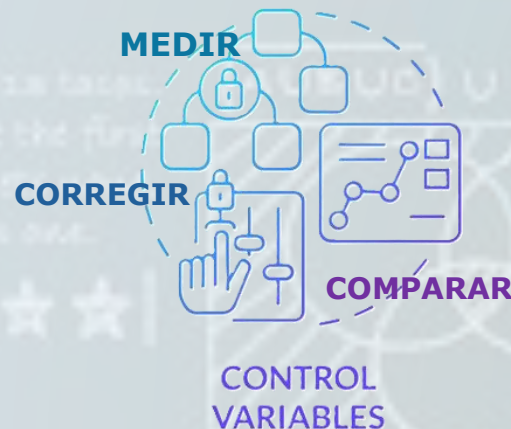
¿qué necesito guardar dentro de la clase para que sus operaciones funcionen correctamente?

ESTRUCTURA

En una implementación simple de una cola, la estructura interna podría ser:

- un arreglo datos
- un entero frente
- un entero fin

*Una estructura necesita saber en qué estado se encuentra. Para eso sirven las **variables de control** que brindan información necesaria para administrar la estructura.*



- ¿la estructura está vacía?
- ¿la estructura está llena?
- ¿dónde debo insertar?
- ¿qué elemento debo devolver?
- ¿qué posición debo actualizar después de quitar?



- Estructura interna:

- Arreglo de datos

- Variable "tope"

- Métodos:

- ➡ `push(x) { ... }`

- ➡ `pop() { ... }`

- ➡ `isEmpty() { ... }`

Si el contrato dice que una pila debe permitir push, pop, peek, isEmpty e isFull, entonces la clase debería ofrecer métodos equivalentes. El contrato funciona como una guía. Le dice a la implementación qué operaciones debe garantizar. La clase, por su parte, transforma esas operaciones en código ejecutable.

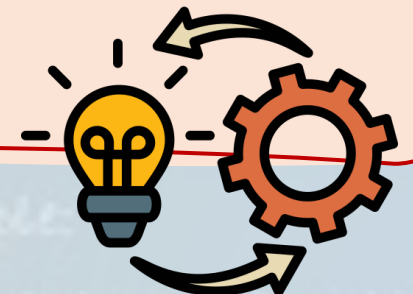


push(x) agrega un elemento al tope.

isEmpty() indica si la estructura no tiene elementos.

Guardar x en la posición tope + 1 del arreglo y luego actualizar tope.

verificar si tope == -1 en la pila, o si frente y fin indican ausencia de elementos en la cola.



En ambos casos, el usuario no ve la mecánica interna. Solo ve el resultado de la operación.



- Estructura interna:
 - Arreglo de datos
 - Variable “tope”
- Métodos:
 - ➡ `push(x) { ... }`
 - ➡ `pop() { ... }`
 - ➡ `isEmpty() { ... }`

Si la estructura interna y las variables de control quedaran expuestas, cualquier parte del programa podría alterarlas sin respetar las reglas del TDA. Por ejemplo:

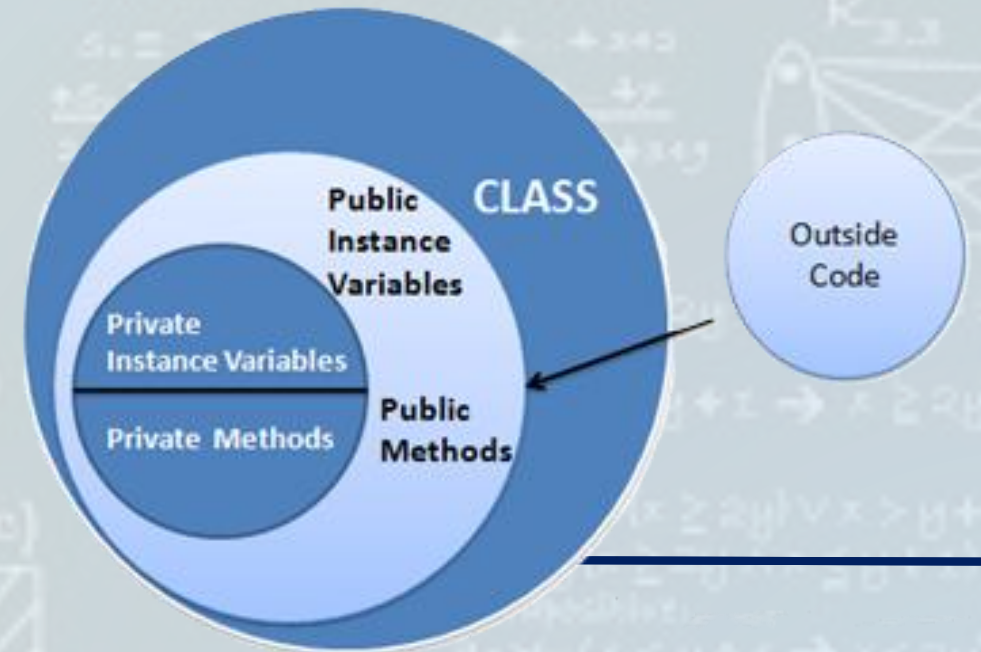
- cambiar el tope de una pila manualmente
- mover el frente de una cola sin quitar realmente un elemento
- escribir directamente en el arreglo ignorando la lógica de inserción

Eso generaría estructuras incoherentes y errores difíciles de detectar.

Encapsulamiento



Los atributos internos suelen declararse como privados, y las operaciones del contrato se ofrecen mediante métodos públicos



La clase conserva el control sobre su propio estado y garantiza que cualquier modificación pase por las reglas correctas



Errores comunes al pasar del contrato al código

Confundir la estructura con el arreglo. El arreglo no es la pila ni la cola; es solo una posible representación.

Centrarse demasiado rápido en el código sin haber entendido bien el contrato. Cuando eso pasa, se implementa métodos, pero no siempre se respeta el comportamiento esperado.

Olvidar actualizar correctamente las variables de control. Por ejemplo, insertar bien un elemento pero no mover el tope, o quitar un dato pero no avanzar el frente.

Por eso, antes de programar, conviene preguntarse siempre:

- ¿qué promete hacer esta operación?
- ¿qué estado interno necesito para lograrlo?
- ¿qué variable de control debo actualizar?
- ¿qué precondition debo verificar?

**¡NO
PROGRAMES
ASÍ!**





```
public class Pila {
    private int[] datos;
    private int tope;
    private int capacidad;

    public Pila(int capacidad) {
        this.capacidad = capacidad;
        this.datos = new int[capacidad];
        this.tope = -1;
    }

    public boolean isEmpty() {
        return tope == -1;
    }

    public boolean isFull() {
        return tope == capacidad - 1;
    }

    public void push(int valor) {
        if (isFull()) {
            System.out.println("Error: la pila está llena.");
            return;
        }
        tope++;
        datos[tope] = valor;
    }

    public int pop() {
        if (isEmpty()) {
            System.out.println("Error: la pila está vacía.");
            return -1;
        }
        int valor = datos[tope];
        tope--;
        return valor;
    }
}
```



PILA - Pasando al código



```
public int peek() {
    if (isEmpty()) {
        System.out.println("Error: la pila está vacía.");
        return -1;
    }
    return datos[tope];
}

public void mostrar() {
    if (isEmpty()) {
        System.out.println("La pila está vacía.");
        return;
    }

    System.out.println("Contenido de la pila:");
    for (int i = tope; i >= 0; i--) {
        System.out.println(datos[i]);
    }
}
```



```
public class MainPila {
    public static void main(String[] args) {
        Pila pila = new Pila(5);

        pila.push(10);
        pila.push(20);
        pila.push(30);

        pila.mostrar();

        System.out.println("Tope actual: " + pila.peek());
        System.out.println("Elemento desapilado: " + pila.pop());

        pila.mostrar();
    }
}
```





```
public class Cola {
    private int[] datos;
    private int frente;
    private int fin;
    private int cantidad;
    private int capacidad;

    public Cola(int capacidad) {
        this.capacidad = capacidad;
        this.datos = new int[capacidad];
        this.frente = 0;
        this.fin = -1;
        this.cantidad = 0;
    }

    public boolean isEmpty() {
        return cantidad == 0;
    }

    public boolean isFull() {
        return cantidad == capacidad;
    }

    public void enqueue(int valor) {
        if (isFull()) {
            System.out.println("Error: la cola está llena.");
            return;
        }
        fin++;
        datos[fin] = valor;
        cantidad++;
    }

    public int dequeue() {
        if (isEmpty()) {
            System.out.println("Error: la cola está vacía.");
            return -1;
        }
        int valor = datos[frente];
        frente++;
        cantidad--;
        return valor;
    }
}
```



COLA - Pasando al código



```
public int front() {
    if (isEmpty()) {
        System.out.println("Error: la cola está vacía.");
        return -1;
    }
    return datos[frente];
}

public void mostrar() {
    if (isEmpty()) {
        System.out.println("La cola está vacía.");
        return;
    }

    System.out.print("Contenido de la cola: ");
    for (int i = frente; i <= fin; i++) {
        System.out.print(datos[i] + " ");
    }
    System.out.println();
}
```



```
public class MainCola {
    public static void main(String[] args) {
        Cola cola = new Cola(5);

        cola.enqueue(10);
        cola.enqueue(20);
        cola.enqueue(30);

        cola.mostrar();

        System.out.println("Frente actual: " + cola.front());
        System.out.println("Elemento desencolado: " + cola.dequeue());

        cola.mostrar();
    }
}
```

Relación entre contrato y código

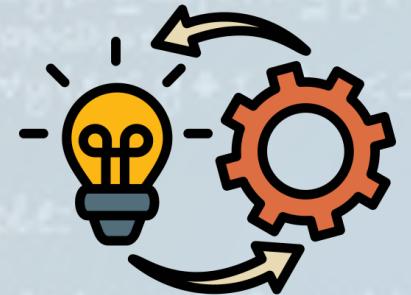


En la pila:

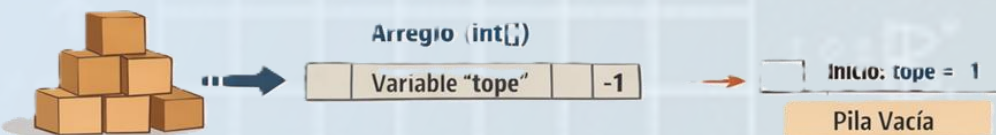
- `push(x) → public void push(int valor)`
- `pop() → public int pop()`
- `peek() → public int peek()`
- `isEmpty() → public boolean isEmpty()`
- `isFull() → public boolean isFull()`

En la cola:

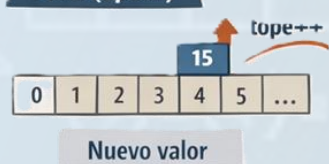
- `enqueue(x) → public void enqueue(int valor)`
- `dequeue() → public int dequeue()`
- `front() → public int front()`
- `isEmpty() → public boolean isEmpty()`
- `isFull() → public boolean isFull()`



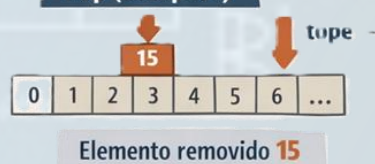
IMPLEMENTACIÓN ESTÁTICA DE UNA PILA



Push (apilar)



Pop (desapilar)



Estado de la Pila

tope = -1 Pila Vacía

tope = Máximo Pila Llena



Control del Tope

- ✓ Antes de hacer push, verificar si hay espacio
- ✓ Antes de hacer pop, verificar si está vacía

“Mover el tope decide todo: Insertar, quitar y validar.”

Implementación estática con arreglos

Cuando decimos que una estructura se implementa de forma estática, queremos decir que su capacidad se define de antemano y permanece fija durante toda la ejecución. En lugar de crecer o reducirse dinámicamente, la estructura trabaja con un bloque de memoria ya reservado. Ese bloque, en este caso, es un arreglo.

COLA

Estructura de Datos Abstracta

Principio FIFO

— Primero en entrar, primero en salir —

ENTRADA → A → B → C → D → SALIDA

— Entra un elemento al final —

→ A → B → C → D → E

— Sale un elemento del frente —

→ B → C → D → E

COLA

Operaciones Básicas

ENQUEUE Agregar un elemento al final de la cola

→ B → C → D → +E → E

DEQUEUE Quitar el elemento del frente de la cola

→ A → B → C → D

FRONT Ver el primer elemento de la cola

Frente: A

Actividad

Instalación de JDK y Eclipse – Mi Primer Proyecto

Para desarrollar en Java es necesario instalar primero el JDK (Java Development Kit), que contiene el compilador y la máquina virtual. Luego se instala un entorno de desarrollo como Eclipse, que facilita la escritura, compilación y ejecución de los programas.

<https://www.oracle.com/java/technologies/downloads/>

https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.msi

○ Eclipse IDE for Java Developers

○ <https://www.eclipse.org/downloads/packages>



Eclipse IDE for Java Developers

352 MB 20,423 DOWNLOADS

The essential tools for any Java developer, including a Java IDE, a Git client, XML Editor, Maven and Gradle integration



Windows: AArch64 | x86_64
macOS: AArch64 | x86_64
Linux: AArch64 | riscv64 | x86_64

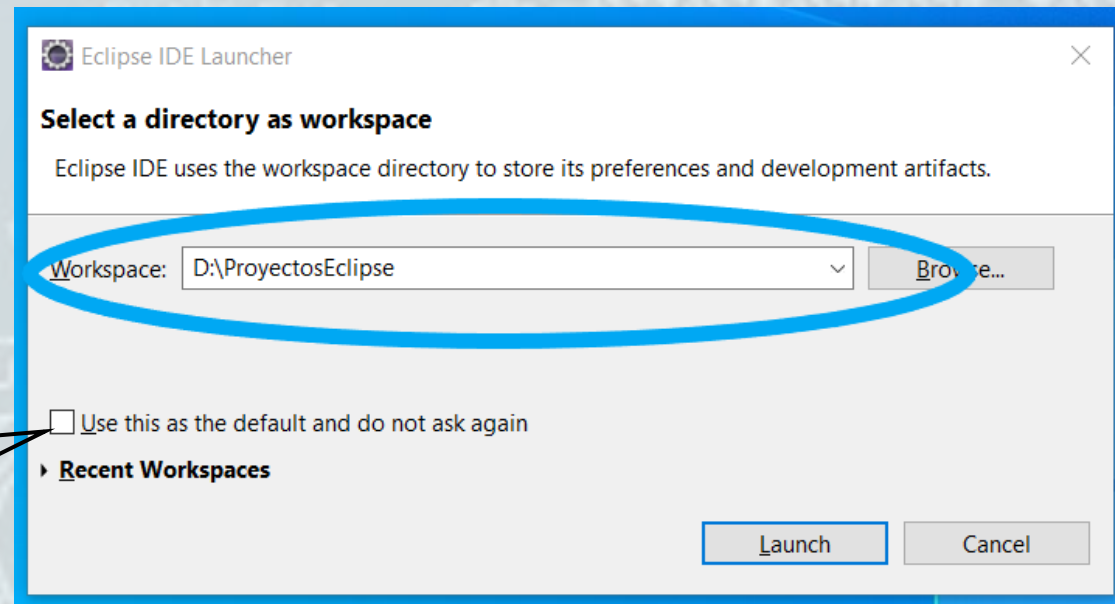
Eclipse IDE for Java Developers



Utiliza concepto de área de trabajo, un directorio donde almacenará los proyectos y configuraciones del editor.

Todos los proyectos en el mismo espacio de trabajo pueden compartir clases y configuraciones.

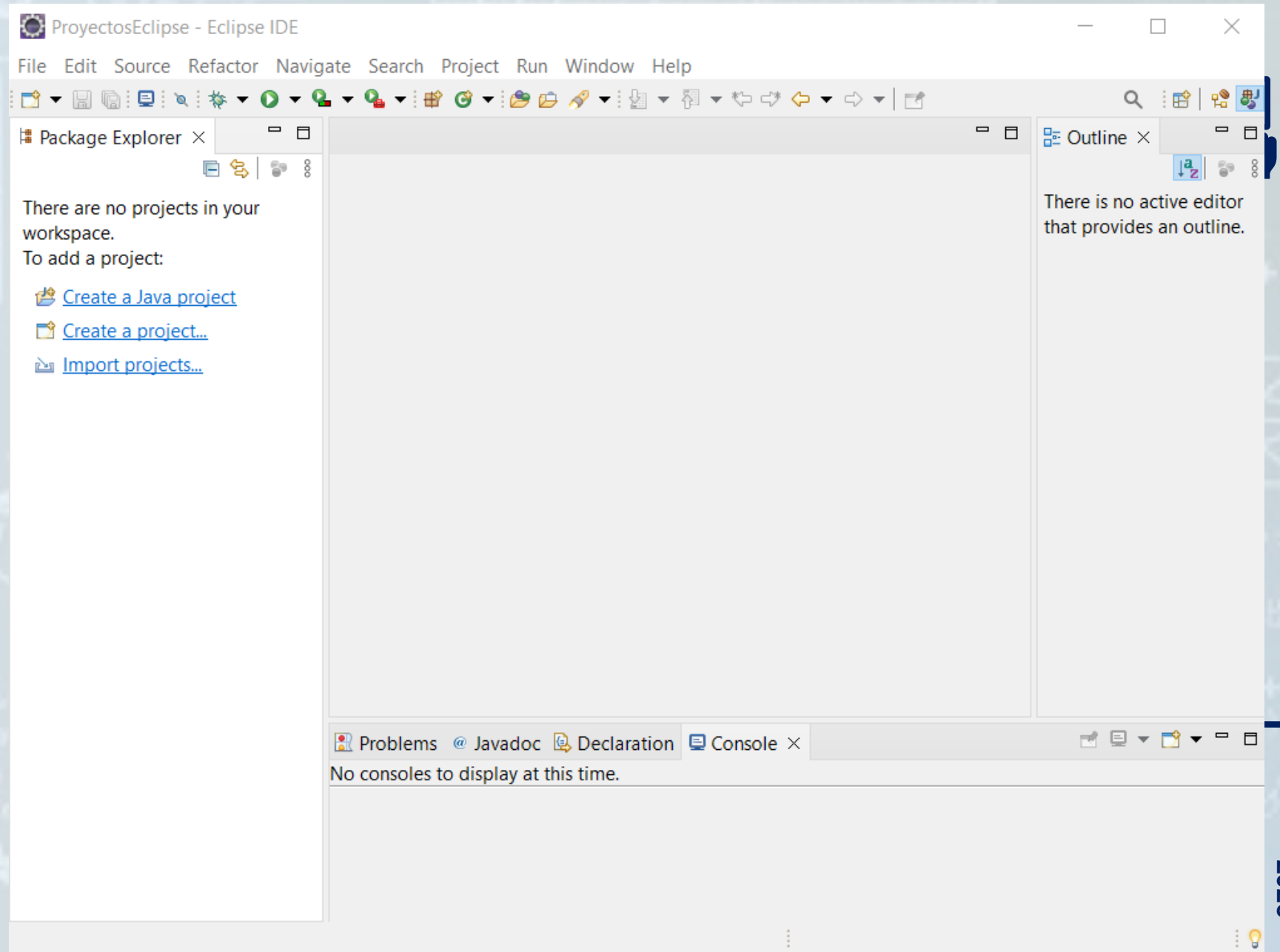
Podemos pedirle que guarde el área de trabajo en la configuración. Esta pregunta no volverá a aparecer



Una vez finalizada la carga del editor, nos pide que definamos un proyecto

Podemos crear uno nuevo o importar uno existente

Siempre hay que crear **proyectos java**



Para crear un nuevo proyecto hacemos...

File->new->java project

Le asignamos un nombre, el mismo debería ayudarnos en la identificación del mismo.

Si no estamos seguros podemos usar los seteos del área de trabajo

Le damos finalizar y ya tenemos el proyecto creado.

New Java Project

Create a Java Project

Enter a project name.

Project name:

☒ Use default location

Location: [Browse...](#)

JRE

☒ Use an execution environment JRE:

☐ Use a project specific JRE:

☐ Use default JRE 'jdk-17.0.2' and workspace compiler preferences [Configure JREs...](#)

Project layout

☐ Use project folder as root for sources and class files

☒ Create separate folders for sources and class files [Configure default...](#)

Working sets

☐ Add project to working sets [New...](#)

Working sets: [Select...](#)

Module

☒ Create module-info.java file

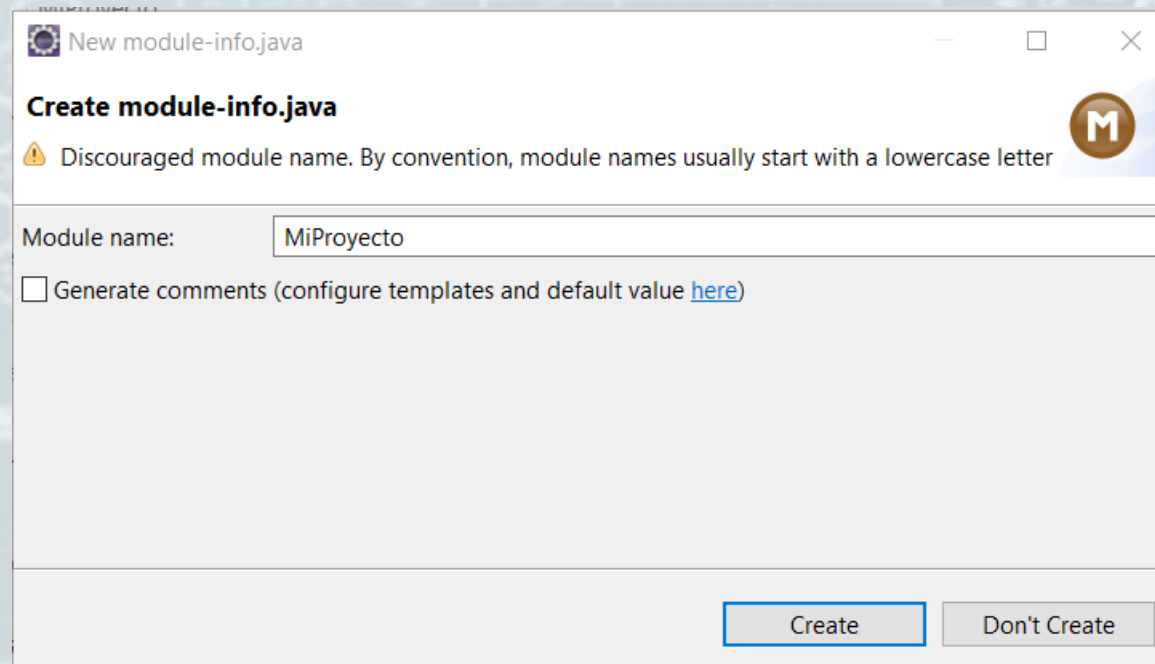
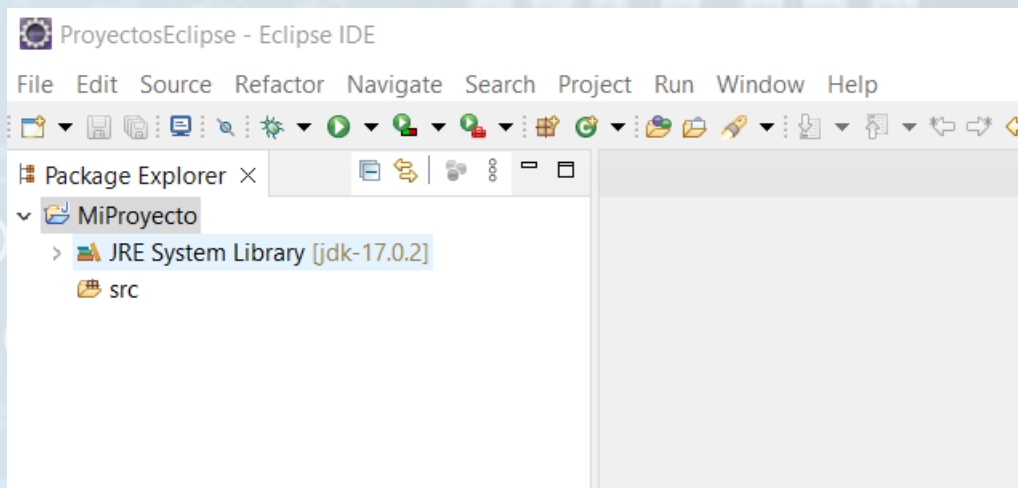
[?](#) [< Back](#) [Next >](#) [Finish](#) [Cancel](#)

Crear carpetas separadas nos ayudara a ordenar nuestro código 

Eclipse IDE for Java Developers



Cuando nos pida crear un modulo de Proyecto, le vamos a decir que no lo cree



Cuando finaliza la creación, deberíamos tener un proyecto con el nombre que le indicamos, adentro encontramos las librerías de java y una carpeta llamada **src** que es donde colocaremos nuestro código.

Eclipse IDE for Java Developers



Que hizo Eclipse en estos pasos?

- Crea una carpeta con el nombre del área de trabajo.
- Coloco una serie de archivos y carpetas de configuración de la IDE.
- Creo una carpeta con el nombre del proyecto
- Creo unos archivos de configuración del proyecto.
- Crea la carpeta **src** para escribir el código fuente.

equipo > Disco local (D:) > ProyectosEclipse			
☐ Nombre	Fecha de modificación	Tipo	Tamaño
.metadata	11/03/2022 19:53	Carpeta de archivos	
☒ MiProyecto	23/03/2022 11:39	Carpeta de archivos	

ste equipo > Disco local (D:) > ProyectosEclipse > MiProyecto	
☐ Nombre	Fecha de modificación
bin	23/03/2022 11:39
src	23/03/2022 11:39
.classpath	23/03/2022 11:39
.project	23/03/2022 11:39

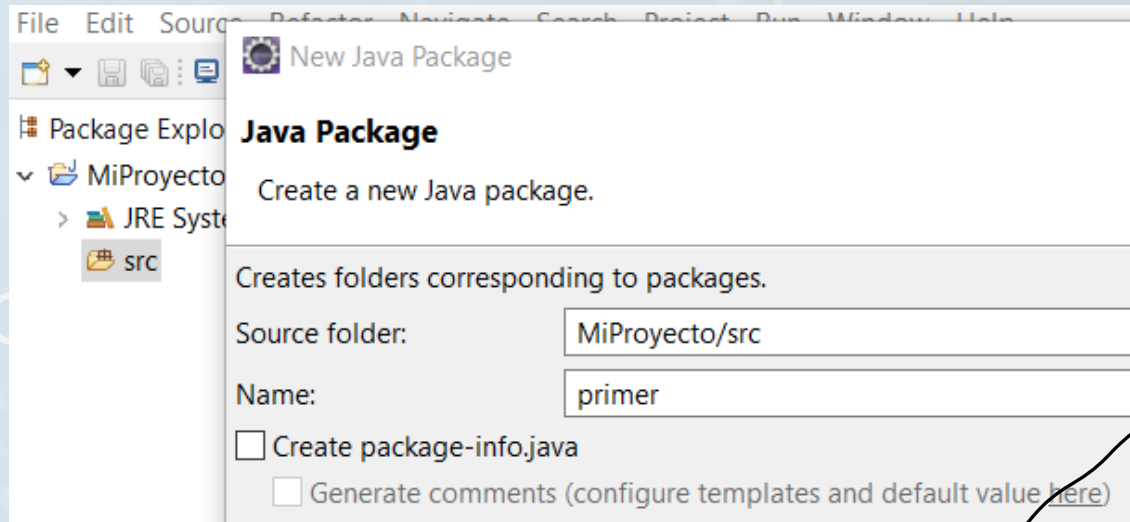
e equipo > Disco local (D:) > ProyectosEclipse > MiProyecto > src			
☐ Nombre	Fecha de modificación	Tipo	Tamaño
Esta carpeta está vacía.			

Eclipse IDE for Java Developers

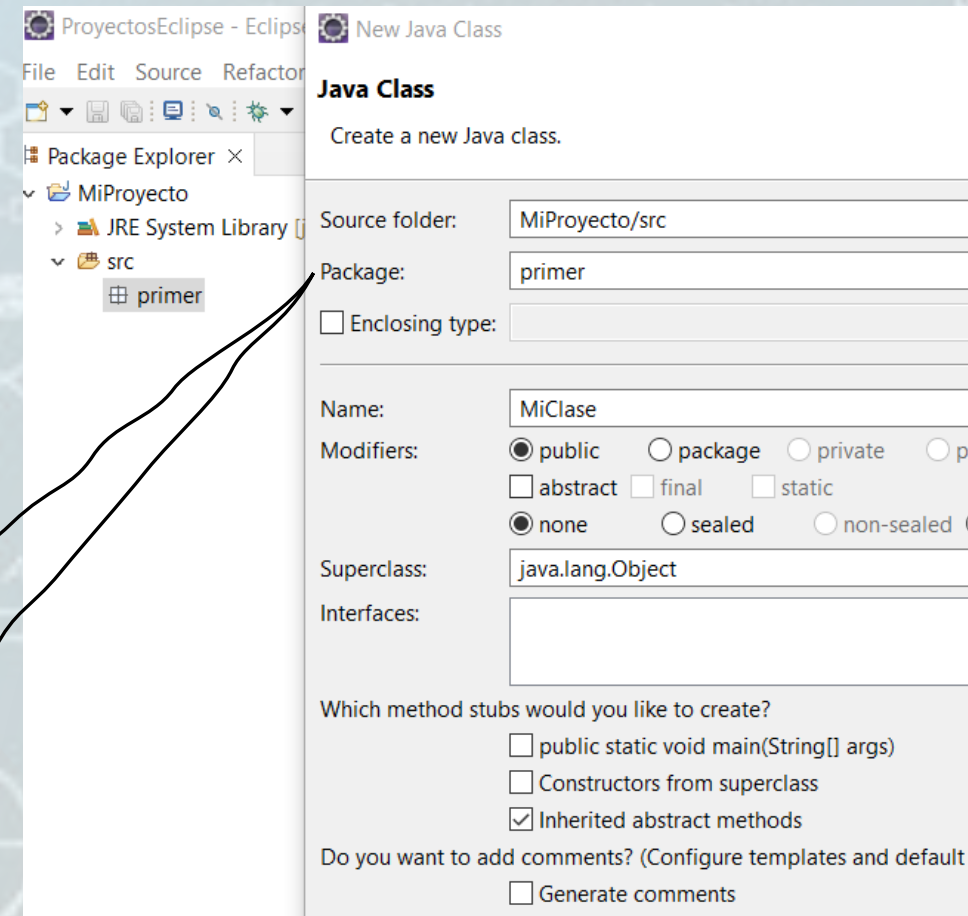


Clases y Paquetes

Paquetes: proporcionan una separación lógica para las clases, los creamos haciendo **file -> new -> package**



Clases: archivos donde escribimos nuestro código. Las creamos haciendo **file -> new -> class**



Eclipse IDE for Java Developers

Una clase en detalle:

Los nombres de las clases comienzan con una letra mayuscula

Modificadores

Herencia

Implementación

Métodos asistidos

Comentarios

New Java Class

Java Class

Create a new Java class.

Source folder: MiProyecto/src Browse...

Package: primer Browse...

☐ Enclosing type: Browse...

Name: MiClase

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Superclass: java.lang.Object Browse...

Interfaces: Add... Remove

Which method stubs would you like to create?
☐ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

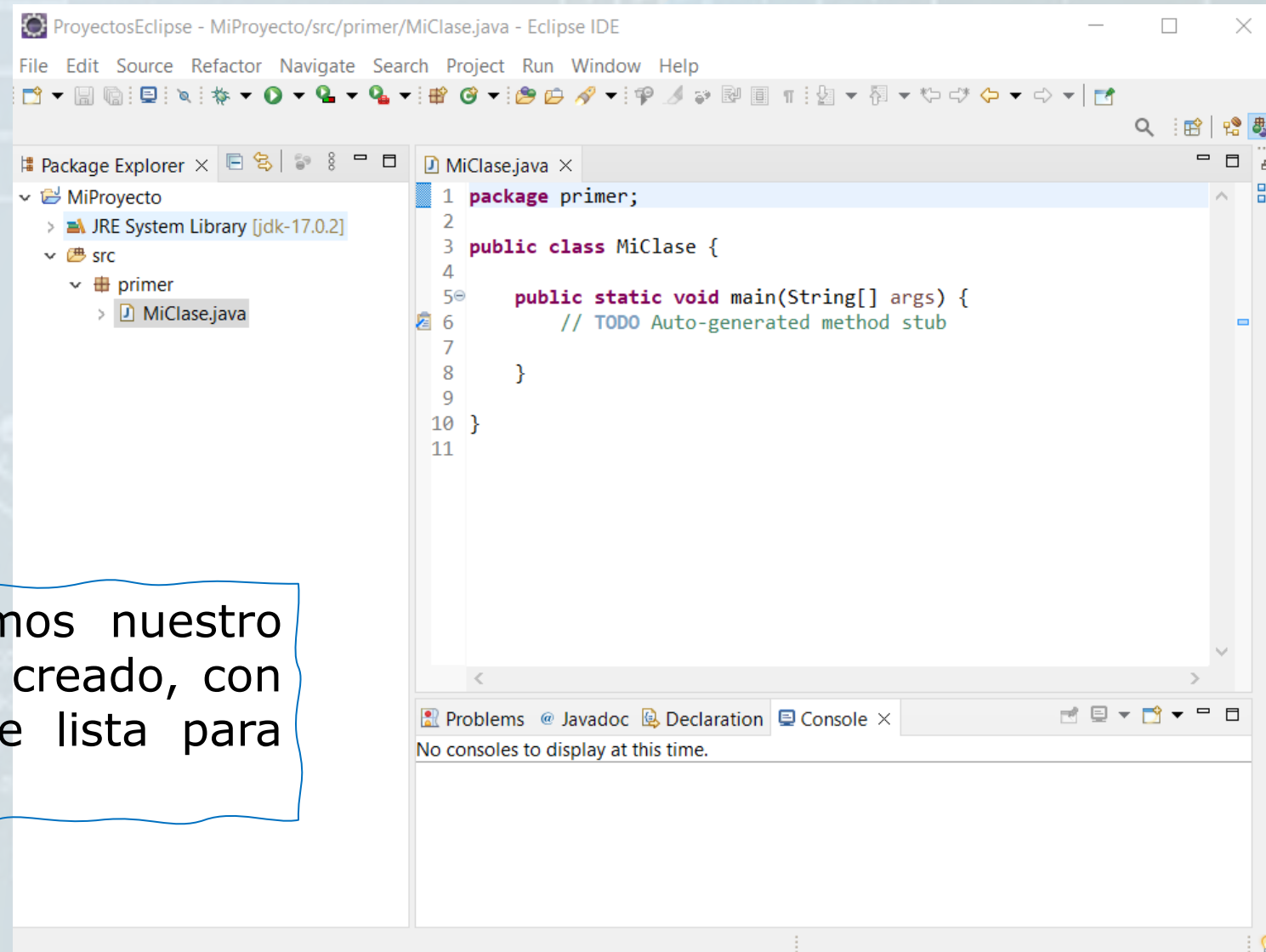
Finish Cancel

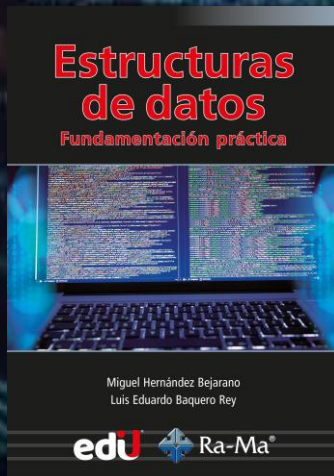
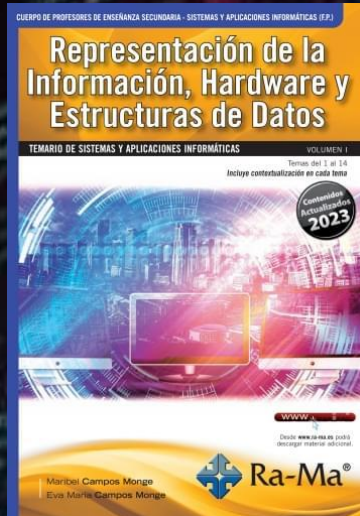


Eclipse IDE for Java Developers



Ya tenemos nuestro Proyecto creado, con una clase lista para ejecutar





BIBLIOGRAFIA



Weiss, M. A. (2014). Data Structures and Algorithm Analysis in Java (3rd ed.). Pearson.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data Structures and Algorithms in Java (6th ed.). Wiley.

Lafore, R. (2003). Data Structures and Algorithms in Java (2nd ed.). Sams Publishing.

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

Oracle. (2023). The Java™ Tutorials.

<https://docs.oracle.com/javase/tutorial/>

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.



PROXIMO ENCUENTRO



Clase 3

Cola con prioridad y Conjunto: elegir estructura según el problema

TDA Cola con Prioridad.

TDA Conjunto.

Estrategias de implementación.